

# AI-Powered Cyber Risk Quantification (CRQ) Platform

A FAIR-based, India-regulation-aware risk quantification and board reporting system

---

## Technical Report

|               |                                                                                                               |
|---------------|---------------------------------------------------------------------------------------------------------------|
| Repository    | <a href="https://github.com/Sid-DudeD017/CRQ_Platform">github.com/Sid-DudeD017/CRQ_Platform</a>               |
| Frontend      | <a href="https://crq-platform.vercel.app">crq-platform.vercel.app</a> (Next.js on Vercel)                     |
| Backend API   | <a href="https://crq-platform-backend.onrender.com">crq-platform-backend.onrender.com</a> (FastAPI on Render) |
| Document date | 29 August 2026                                                                                                |

# Table of Contents

|                                                                |    |
|----------------------------------------------------------------|----|
| Table of Contents                                              | 1  |
| 1. Introduction & Problem Statement                            | 3  |
| 2. Solution Overview                                           | 3  |
| 3. System Architecture                                         | 4  |
| 3.1 Layer Responsibilities . . . . .                           | 4  |
| 4. Technology Stack                                            | 6  |
| 5. Core Quantitative Models                                    | 7  |
| 5.1 FAIR Loss Pipeline . . . . .                               | 7  |
| 5.2 Network-Effective Vulnerability (“Blast Radius”) . . . . . | 7  |
| 5.3 Control Strength Deduction Model . . . . .                 | 8  |
| 5.4 DPDP Act Contextual Penalty Adjustment . . . . .           | 8  |
| 5.5 SEBI Cyber Capability Index (CCI) . . . . .                | 8  |
| 5.6 Security Budget Optimizer & ROSI . . . . .                 | 8  |
| 6. Data Model                                                  | 10 |
| 7. API Reference                                               | 10 |
| 8. Security Architecture                                       | 11 |
| 9. Regulatory Compliance Mapping                               | 11 |
| 10. Deployment Architecture                                    | 12 |
| 11. Testing & Quality Assurance                                | 12 |
| 12. Known Limitations                                          | 13 |

|                       |           |
|-----------------------|-----------|
| <b>13. Roadmap</b>    | <b>13</b> |
| <b>14. Conclusion</b> | <b>13</b> |

# 1. Introduction & Problem Statement

Boards and CFOs approve security budgets and accept residual risk in rupees, while security teams report in CVSS scores, patch counts and threat-intel feeds. The two vocabularies rarely connect, which makes it hard to answer the one question a board actually asks: *how much rupee exposure does the organisation carry today, and does the proposed spend reduce it enough to justify the cost?*

India's regulatory environment sharpens this gap further. The RBI Cyber Security Framework expects board-level oversight of every material risk-acceptance decision. SEBI's Cyber Security and Cyber Resilience Framework (CSCRF, August 2024) requires regulated entities to score their own resilience across five capability pillars. The Digital Personal Data Protection Act, 2023 (DPDP) attaches statutory penalties reaching into the hundreds of crores for mishandled personal data. None of these obligations are naturally expressed in the language a SOC dashboard speaks.

The CRQ Platform closes that gap. It ingests live security telemetry (vulnerability scans, SIEM, IAM, EDR, CSPM signals) and CMDB asset data, runs it through a FAIR-based Monte Carlo loss model to produce an Annualized Loss Expectancy in rupees, layers India-specific compliance scoring on top (SEBI CCI, DPDP triggers), and optimizes a proposed security budget against that model with a provable-optimal knapsack solver — then lets an authorised executive accept the residual risk through a workflow that writes an auditable record of the decision.

## 2. Solution Overview

*What the platform actually does, end to end.*

- **Telemetry & CMDB ingestion** — assets, vulnerability scans, SIEM/IAM/EDR/CSPM signals, and a network topology graph, either seeded for demonstration or ingested from real tools.
- **FAIR-based Monte Carlo risk quantification** — 10,000-iteration simulation translating threat frequency, vulnerability, and loss magnitude into an Annualized Loss Expectancy (ALE) and Value-at-Risk in rupees.
- **Network-aware (“blast radius”) vulnerability propagation** — a compromised or exposed asset raises the effective risk of everything connected to it, not just itself.
- **SEBI Cyber Capability Index scoring** — the same simulation run is re-expressed against SEBI CSCRF's five resilience pillars (Anticipate, Withstand, Contain, Recover, Evolve).
- **DPDP Act-aware loss modelling** — exposure involving PII-classified assets above a vulnerability threshold automatically pulls in DPDP's statutory penalty ceiling.
- **Provable-optimal budget allocation** — a 0/1 knapsack solve (not a greedy heuristic) picks the set of security controls that maximizes risk reduction under a given budget.
- **Blockchain-backed audit ledger** — every board-approved risk-acceptance decision is written to a smart contract, giving RBI-mandated oversight a tamper-evident trail.
- **Virtual CISO assistant** — a LangGraph multi-agent system, grounded by retrieval over real compliance-framework text, answers questions about the live simulation in natural language.
- **Executive dashboard** — role-based views (CISO / CFO) for running simulations, approving optimizer plans, accepting risk, and pulling board-ready reports.

### 3. System Architecture

Six layers, one entrypoint. The FastAPI gateway is the only thing the frontend, the quant engine, the AI agent, and the blockchain client ever talk to directly — nothing reaches the database or the smart contract except through it.

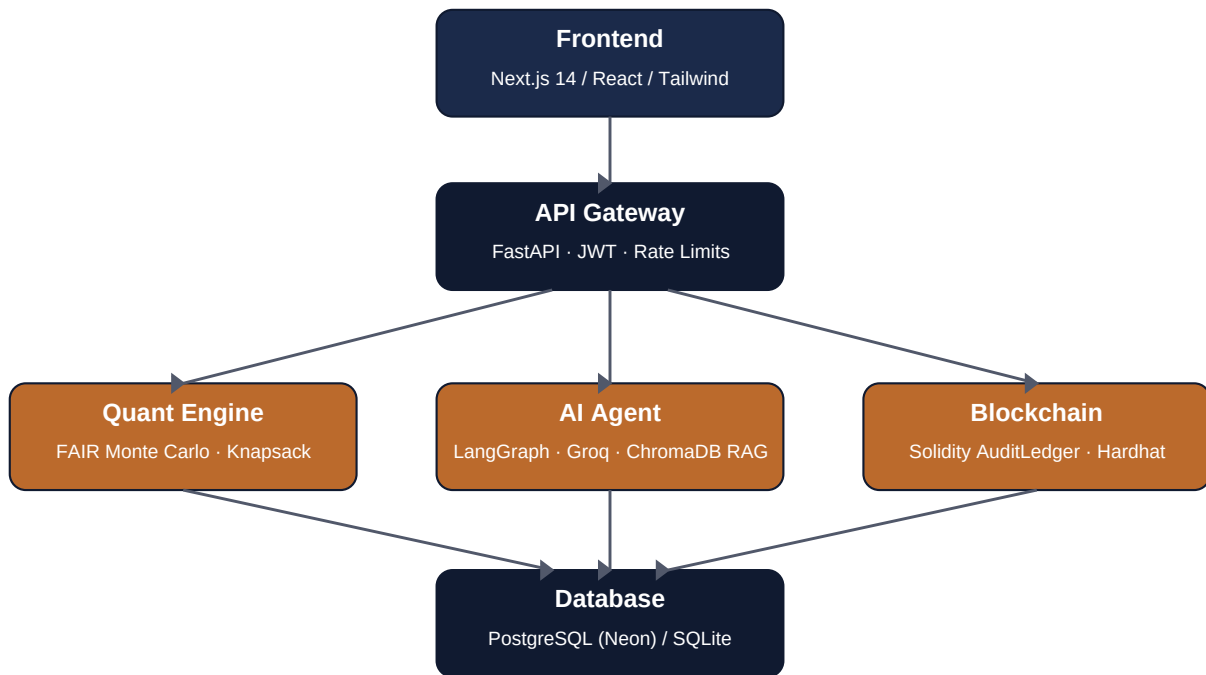


Figure 3.1 — CRQ Platform layer architecture and data flow

#### 3.1 Layer Responsibilities

| Layer        | Responsibility                                                                                                                                          |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frontend     | Executive dashboard, Investment Optimizer, Ledger, Reports, Docs, chat widget. Talks to the backend only over HTTP.                                     |
| API Gateway  | Single FastAPI endpoint: auth, CMDB/telemetry endpoints, simulation orchestration, chat streaming, audit logging.                                       |
| Quant Engine | Pure-Python risk math: FAIR Monte Carlo loss simulation and the 0/1 knapsack budget optimizer.                                                          |
| AI Agent     | LangGraph multi-agent orchestrator (security analyst / compliance officer / quant analyst / supervisor) with a RAG tool over compliance-framework text. |
| Blockchain   | Solidity AuditLedger.sol contract; web3.py client commits each board-approved decision on-chain.                                                        |
| Database     | PostgreSQL in production (Neon), SQLite in local development — selected automatically from DATABASE_URL.                                                |

## 4. Technology Stack

| Area               | Technology                                          | Role                                                  |
|--------------------|-----------------------------------------------------|-------------------------------------------------------|
| Frontend framework | Next.js 14 (App Router), React 18, TypeScript       | Executive dashboard and all sub-pages                 |
| Styling            | Tailwind CSS with Material Design 3 semantic tokens | Theming, light/dark mode, consistent spacing scale    |
| Charts             | Recharts                                            | Loss-distribution curve, efficiency frontier          |
| API gateway        | FastAPI, Pydantic, Uvicorn                          | HTTP routing, request validation, ASGI server         |
| ORM / DB layer     | SQLModel (SQLAlchemy + Pydantic)                    | Asset / Telemetry / Simulation / Decision models      |
| Database           | PostgreSQL (Neon, serverless) / SQLite (dev)        | Persistent storage, selected via DATABASE_URL         |
| Auth               | PyJWT, OAuth2PasswordBearer                         | Bearer-token auth for exec accounts                   |
| Rate limiting      | slowapi                                             | Per-IP limits on the expensive/public endpoints       |
| Quant engine       | NumPy, SciPy (gaussian_kde)                         | Monte Carlo simulation and loss-distribution density  |
| Optimizer          | PuLP (CBC solver)                                   | Exact 0/1 knapsack solve for budget allocation        |
| AI orchestration   | LangGraph, LangChain                                | Multi-agent supervisor / analyst graph                |
| LLM                | Groq (langchain-groq), model openai/gpt-oss-20b     | Virtual CISO natural-language responses               |
| RAG store          | ChromaDB                                            | Vector search over RBI/SEBI/NIST/DPDP reference text  |
| Smart contract     | Solidity                                            | AuditLedger.sol — on-chain risk-decision record       |
| Blockchain tooling | Hardhat, web3.py                                    | Local chain, contract deployment, transaction signing |
| Testing            | pytest, FastAPI TestClient                          | Automated endpoint and regression coverage            |
| Hosting — frontend | Vercel                                              | Next.js deployment, auto-deploy from GitHub           |
| Hosting — backend  | Render (free tier, Blueprint)                       | FastAPI deployment, auto-deploy from GitHub           |
| Hosting — database | Neon                                                | Managed serverless PostgreSQL                         |

## 5. Core Quantitative Models

Every number the dashboard shows traces back to one of the models below. Sections 5.1–5.4 form a pipeline — each stage's output is the next stage's input — computed fresh on every POST /api/simulate-risk call from whatever telemetry is currently in the database.

### 5.1 FAIR Loss Pipeline

Run 10,000 times per request in `quant-engine/monte_carlo.py`.

#### Step 1 — Threat Event Frequency (TEF)

```
tef = triangular(tef_min, tef_mode, tef_max)

base_tef = 10 + high_threat_count * 5
          + sum(event_frequency_24h * 0.001)
          + sum(anomalous_access_flags * 2)
          + 50 if cisa_kev_presence
          + 20 if incident_alert_level == "CRITICAL"

tef_min, tef_mode, tef_max = max(5, base_tef - 20), base_tef, base_tef + 50
```

#### Step 2 — Vulnerability (probability a threat becomes a loss)

```
threat_capability = triangular(20, 60, 95)
control_strength  = triangular(cs_min, cs_mode, cs_max)      # see 5.3

vulnerability = U(0.6, 0.9) if threat_capability > control_strength
               else U(0.1, 0.4)
```

#### Step 3 — Loss Event Frequency & Loss Magnitude

```
lef = tef * vulnerability

base_plm = total_business_value * 0.05
base_slm = total_business_value * 0.02
primary_loss  = triangular(0.5*base_plm, base_plm, 2*base_plm)
secondary_loss = triangular(0.5*base_slm, base_slm, 2*base_slm) # or DPDP-adjusted, see 5.4
```

#### Step 4 — Annualized Loss Expectancy (ALE)

```
annual_loss = lef * (primary_loss + secondary_loss)

mean_expected_loss = mean(annual_loss)
var_95              = percentile(annual_loss, 95)
var_99              = percentile(annual_loss, 99)
distribution_curve  = gaussian_kde(annual_loss) # 100-point density curve
```

### 5.2 Network-Effective Vulnerability (“Blast Radius”)



Computed before the FAIR call so a well-patched asset sitting next to compromised neighbours still shows up as exposed — this feeds Control Strength in 5.3.

```
v_eff[i] = v_intrinsic[i] + 0.2 * sum(adjacency[i][j] * v_intrinsic[j] for all j)

avg_vulnerability = mean(v_eff)
avg_cs_upstream   = max(10, 100 - avg_vulnerability * 10)
```

### 5.3 Control Strength Deduction Model

```
deductions += 5.0   if not mfa_active
deductions += 2.0   if excessive_permissions
deductions += cvss_score (or 10.0)   if patch_status == "Missing Critical"
deductions += 4.0   if edr_health_status != "Healthy"
deductions += 20.0  if host_compromise_flags
deductions += 15.0  if public_exposure_flag
deductions += cloud_misconfigurations_count * 1.0

avg_cs = max(5, avg_cs_upstream - mean(deductions per log))
```

### 5.4 DPDP Act Contextual Penalty Adjustment

```
is_dpdp_applicable = any(
    asset.data_classification == "PII" and v_eff[asset] > 7.0
)

dpdp_penalty_limit = Rs 250,00,00,000 + Rs 200,00,00,000      # Rs 450 Cr ceiling
slm_max_adjusted   = max(slm_max, dpdp_penalty_limit)
slm_mode_adjusted  = min(slm_mode + dpdp_penalty_limit * 0.05, slm_max_adjusted)
```

### 5.5 SEBI Cyber Capability Index (CCI)

The same simulation output, re-expressed against SEBI CSCRF's five resilience pillars, each clamped to a 0–5 scale.

```
cci_anticipate = clamp((100 - mean(tef)) / 20, 0, 5)
cci_withstand  = clamp(mean(control_strength) / 20, 0, 5)
cci_contain    = clamp(5 * (Rs 1,00,00,000 / mean(secondary_loss)), 0, 5)
cci_recover    = clamp(5 * (Rs 50,00,000 / mean(primary_loss)), 0, 5)

cci_score = mean(anticipate, withstand, contain, recover)
cci_evolve = clamp(cci_score * 1.1, 0, 5)
```

### 5.6 Security Budget Optimizer & ROSI

An exact 0/1 knapsack solve via PuLP's CBC solver — not a greedy heuristic — so it finds the true optimum.

```
maximize    sum(risk_reduction[i] * x[i])
subject to  sum(cost[i] * x[i])  <=  budget
            x[i] in {0, 1}

rosi_pct = (risk_reduction - cost) / cost * 100    # display-only, computed client-side
```

## 6. Data Model

| Entity         | Key Fields                                                                                                                                                                                                              |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Asset          | id (e.g. AST-001), name, asset_type, business_unit, business_value, ip_address, data_classification, business_criticality                                                                                               |
| TelemetryLog   | asset_id, timestamp, vulnerability_score, threat_level, cve_ids, cvss_score, patch_status, mfa_active, edr_health_status, host_compromise_flags, public_exposure_flag, cloud_misconfigurations_count, cisa_kev_presence |
| NetworkEdge    | source_asset_id, target_asset_id, weight — used for blast-radius propagation                                                                                                                                            |
| RiskSimulation | expected_annual_loss, var_95, monte_carlo_distribution, optimized_budget_allocation, budget_used                                                                                                                        |
| RiskDecision   | action, risk_accepted, decided_by, board_approved, tx_hash, on_chain, created_at                                                                                                                                        |

## 7. API Reference

| Method | Endpoint                | Description                                                                      |
|--------|-------------------------|----------------------------------------------------------------------------------|
| GET    | /                       | Health check / API gateway greeting                                              |
| POST   | /api/auth/login         | Demo login — returns a bearer token for the CISO/CFO accounts                    |
| POST   | /api/generate-mock-data | Seeds assets, telemetry, and network topology. Requires auth.                    |
| GET    | /api/telemetry          | Latest 100 telemetry log rows                                                    |
| GET    | /api/topology           | Network adjacency matrix and node mapping                                        |
| POST   | /api/simulate-risk      | Runs the FAIR Monte Carlo simulation and budget optimizer. Rate-limited.         |
| POST   | /api/chat               | Streams a response from the Virtual CISO LangGraph agent. Rate-limited.          |
| POST   | /api/audit              | Logs a risk-acceptance decision and fires the blockchain webhook. Requires auth. |
| GET    | /api/audit-log          | Lists every risk-acceptance decision, most recent first                          |

## 8. Security Architecture

- **Authentication** — JWT bearer tokens (PyJWT, HS256) issued against a fixed set of demo executive accounts; every write to the audit trail requires a valid token.
- **Authorization on write paths** — the destructive mock-data reseed endpoint and the audit-logging endpoint both require authentication; read-only simulation and chat stay open for demo accessibility but are rate-limited instead.
- **Rate limiting** — slowapi caps the unauthenticated, compute-heavy endpoints (10/min on simulate-risk, 15/min on chat) per client IP, with `--proxy-headers` enabled on the Render deployment so the real visitor IP is used rather than the platform's proxy address.
- **CORS** — an explicit origin allowlist (the deployed frontend domain and localhost for development) rather than a wildcard.
- **Secrets management** — JWT signing secret, database URL, and LLM API key are all environment-provided (Render's `generateValue` for the signing secret in production), never hardcoded.
- **Audit immutability** — board-approved risk decisions are additionally committed to a Solidity smart contract, giving the RBI-mandated oversight trail a tamper-evident backing beyond the relational database.

## 9. Regulatory Compliance Mapping

| Framework                    | Requirement                                                  | How the Platform Addresses It                                                                             |
|------------------------------|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| RBI Cyber Security Framework | Board/audit-committee oversight of risk-acceptance decisions | <code>board_approved</code> flag recorded on every decision, both in the database and on-chain            |
| SEBI CSCRF (Aug 2024)        | Five-pillar cyber resilience self-assessment                 | SEBI Cyber Capability Index, computed live from the same telemetry driving the FAIR model                 |
| DPDP Act, 2023               | Statutory penalty exposure for personal-data mishandling     | Contextual trigger on PII assets with high effective vulnerability pulls in the Rs 450 Cr penalty ceiling |
| NIST CSF 2.0                 | Identify / Protect / Detect / Respond / Recover coverage     | Telemetry-to-function mapping underlying the Control Strength deduction terms                             |

## 10. Deployment Architecture

The hosted demo runs fully decoupled: the frontend on Vercel, the backend on Render, and the database on Neon's serverless Postgres — connected purely over HTTPS and a Postgres connection string, with no shared filesystem or private network between them.

| Component              | Platform                      | Notes                                                                                                   |
|------------------------|-------------------------------|---------------------------------------------------------------------------------------------------------|
| Frontend               | Vercel                        | Auto-deploys from GitHub main; NEXT_PUBLIC_API_URL points at the Render backend                         |
| Backend                | Render (Blueprint, free tier) | render.yaml declares the service; DATABASE_URL / GROQ_API_KEY set as secrets, SECRET_KEY auto-generated |
| Database               | Neon (serverless PostgreSQL)  | DATABASE_URL env var selects Postgres in production, falls back to local SQLite in dev                  |
| Blockchain audit trail | Local only (Hardhat)          | Not part of the hosted demo — decisions still persist to the database without an on-chain tx            |

## 11. Testing & Quality Assurance

An automated pytest suite (`tests/test_api.py`) exercises the FastAPI gateway end to end using its TestClient against a throwaway SQLite database, so it never touches production data or the live LLM quota.

- Health check and authentication (accepts valid demo credentials, rejects invalid ones).
- The mock-data reseed endpoint correctly rejects anonymous callers and succeeds for an authenticated one.
- Telemetry and topology endpoints return data after seeding.
- **Regression guard:** every SEBI resilience pillar and the composite score stay within [0, 5] regardless of input — this specifically catches the clamping bug found and fixed during development.
- **Regression guard:** the simulate-risk rate limit actually returns 429 once exceeded.
- The full audit flow — log a decision, then confirm it appears in the audit log.

## 12. Known Limitations

- The blockchain audit trail requires a local Hardhat node and is not part of the hosted demo; decisions still persist to the database without an on-chain transaction hash in that case.
- The Overview page's budget slider display and the value it actually sends to the API are on different scales — a display-only mismatch, not a calculation error.
- Several model constants (the 0.2 blast-radius decay factor, the DPDP mode's 5% nudge, the SEBI Contain/Recover reference thresholds) are engineering judgement calibrated against the demo dataset rather than values cited from a named source.
- Training-module completion tracking is documented (see the in-app Training page) but not yet backed by a database table or endpoint.

## 13. Roadmap

- Per-employee training completion records feeding back into the FAIR Control Strength input, the same way EDR/CSPM telemetry does today.
- Optional live blockchain audit trail (testnet) for the hosted deployment, for organisations that want on-chain proof without running their own node.
- Direct connectors to real vulnerability scanners, SIEM, and CSPM tools in place of the mock-data generator.
- Calibrating the model constants flagged in Section 12 against real incident-cost data rather than demo-tuned values.

## 14. Conclusion

The CRQ Platform demonstrates that FAIR-style quantitative risk modelling and India's specific regulatory requirements are not competing frameworks — the same Monte Carlo simulation that produces a rupee-denominated Annualized Loss Expectancy for a CFO can, with a modest re-projection layer, also produce the SEBI Cyber Capability Index and trigger DPDP-aware loss modelling for the same underlying telemetry. Combined with a provably optimal budget allocator and a tamper-evident audit trail for the board's own decisions, it closes the loop from raw security signal to an accountable, regulator-legible financial decision.